

Low-level design

Low-level design (LLD) is a component-level design process that follows a step-by-step refinement process. This process can be used for designing data structures, required software architecture, source code and ultimately, performance algorithms. Overall, the data organization may be defined during requirement analysis and then refined during data design work. Post-build, each component is specified in detail.^[1]

The LLD phase is the stage where the actual software components are designed.

During the detailed phase the logical and functional design is done and the design of application structure is developed during the high-level design phase.

Design phase

A design is the order of a system that connects individual components. Often, it can interact with other systems. Design is important to achieve high reliability, low cost, and good maintain-ability.^[2] We can distinguish two types of program design phases:

- Architectural or high-level design
- Detailed or low-level design

Structured flow charts and HIPO diagrams typify the class of software design tools and these provide a high-level overview of a program. The advantages of such a design tool are that it yields a design specification understandable to non-programmers and provides a good pictorial display of the module dependencies.

A disadvantage is that it may be difficult for software developers to go from a graphic-oriented representation of software design to implementation. Therefore, it is necessary to provide little insight into the algorithmic structure describing procedural steps to facilitate the early stages of software development, generally using Program Design Languages (PDLs).^[3]

Purpose

The goal of LLD or a low-level design document (LLDD) is to give the internal logical design of the actual program code. Low-level design is created based on the high-level design. LLD describes the class diagrams with the methods and relations between classes and program specs. It describes the modules so that the programmer can directly code the program from the document.

A good low-level design document makes the program easy to develop when proper analysis is utilized to create a low-level design document. The code can then be developed directly from the low-level design document with minimal debugging and testing. Other advantages include lower cost and easier maintenance.

References

1. Pressman, Roger S. (2005). *Software Engineering: A Practitioner's Approach* (<https://books.google.com/books?id=bL7QZHtWvaUC>). Palgrave Macmillan. ISBN 978-0-07-301933-8.
 2. Bell, Doug; Morrey, Ian; Pugh, John R. (1997). *The Essence of Program Design* (<https://books.google.com/books?id=893K9g5JNn8C>). Prentice Hall. ISBN 978-0-13-367806-2.
 3. Survey of Program Design Languages (PDLs): Brian A. Nejmeh, Herbert E. Dunsmore
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Low-level_design&oldid=1268136309"